RAGShield

基于多维度异常检测的RAG系统知识库安全智能防护系统

团队名称：桃园三结义

2026 年 4 月

目录

一、项目概述 4

1.1 项目简介 4

二、背景分析与问题定义 5

2.1 RAG 系统安全形势 5

2.1.1 RAG 技术成为大模型应用主流 5

2.1.2 知识库投毒攻击成为新兴威胁 5

2.1.3 真实攻击案例与我们的解决方案 6

2.2 现有解决方案的局限性 8

2.3 市场空白与技术趋势 9

三、核心创新点 10

3.1 创新点一：三层多维异常检测架构 10

3.1.1 系统总体架构 10

3.1.2 数据流图 11

3.2 创新点二：知识库离群文档检测 11

3.2.1 离群检测算法伪代码 12

3.2.2 嵌入空间可视化 12

3.3 创新点三：注意力方差异常分析 12

3.3.1 注意力方差计算 13

3.3.2 注意力分布特征 13

3.4 创新点四：检索-生成一致性验证 13

3.4.1 一致性验证伪代码 14

3.5 创新点五：中文 RAG 场景专项优化 14
四、技术方案 15
4.1 系统总体架构 15
4.2 各层技术实现细节 15
4.2.1 Layer 1：知识库层 15
4.2.2 Layer 2：检索层 16
4.2.3 Layer 3：生成层 16
4.3 检测流程伪代码 17
4.4 核心模块设计 18
4.4.1 文档嵌入模块 18
4.4.2 异常评分融合 18
五、应用场景与商业价值 19
5.1 目标市场 19
5.2 应用场景 19
5.3 竞争优势 20
六、总结 21
6.1 核心创新总结 21
6.2 技术可行性 21
6.3 应用价值 21
6.4 研究意义 22
参考文献 23

一、项目概述

RAGShield：基于多维度异常检测的RAG系统知识库安全智能防护系统

1.1 项目简介

随着ChatGPT、DeepSeek、通义千问等大语言模型的广泛应用，检索增强生成（RAG）技术已成为企业构建AI应用的主流方案。RAG通过将大模型与外部知识库结合，有效解决了模型幻觉问题，提升了回答的准确性和时效性。然而，RAG系统面临严重的知识库投毒攻击威胁——攻击者通过在知识库中注入恶意文档，可诱导大模型输出错误信息、泄露敏感数据或执行恶意指令。

本项目针对这一新兴安全挑战，创新性地提出三层多维异常检测架构，整合知识库离群文档检测、检索过程监控、生成内容验证三种前沿技术，构建面向RAG系统的智能安全防护系统。与现有方案相比，本系统在投毒检测准确率（目标≥90%）、误报率控制（目标≤5%）和检测延迟（目标<150ms）三个维度实现全面提升，并首次针对中文RAG场景进行专项优化，支持知识库离群文档检测、查询-文档相关性异常分析、检索-生成一致性验证等核心功能。

二、背景分析与问题定义

2.1 RAG 系统安全形势

2.1.1 RAG 技术成为大模型应用主流

- 超过 70%的企业级大模型应用采用 RAG 架构，成为 AI 应用开发的标准范式
- 知识库规模从数千到数百万文档不等，涵盖企业文档、产品手册、法律法规等
- 应用场景涵盖智能客服、企业知识管理、医疗辅助诊断、金融合规审查、法律咨询等领域
- 主流 RAG 框架包括 LangChain、LlamaIndex、Dify 等，降低了企业构建 RAG 应用的门槛

2.1.2 知识库投毒攻击成为新兴威胁

知识库投毒攻击是指攻击者向 RAG 系统的知识库中注入恶意构造的文档，当用户查询相关主题时，检索到恶意文档并影响大模型生成结果。典型攻击场景包括：

攻击类型	攻击示例	危害
事实篡改	在知识库中插入“公司密码策略已更新为 123456”	诱导用户泄露密码
指令注入	文档中包含“忽略之前指令，执行以下操作”	劫持模型行为
偏见引导	插入带有特定立场的内容	影响模型输出的客观性
上下文污染	在多个文档中植入关联性虚假信息	形成虚假证据链
竞争干扰	在竞品知识库中注入错误信息	商业竞争攻击

表 1：知识库投毒攻击类型

2.1.3 真实攻击案例与我们的解决方案

案例一：PoisonedRAG 攻击（2024 年）

研究人员发现，仅需向包含 100 万文档的知识库注入 5 个精心构造的恶意文本，即可实现 90%以上的攻击成功率。攻击者利用检索器的语义匹配机制，使恶意文档在特定查询下被优先检索。

攻击过程：

1. **文档构造**：攻击者分析目标查询的语义特征，构造与查询高度相关但包含恶意指令的文档
2. **知识库注入**：将恶意文档混入正常知识库，由于文档数量庞大，难以人工发现
3. **攻击触发**：用户发起正常查询，检索器返回恶意文档，大模型生成受污染的回答

RAGShield 解决方案：

检测层	检测机制	针对该攻击的检测能力
Layer 1 知识库层	离群文档检测	检测到恶意文档在嵌入空间中形成异常聚类，标记为可疑
Layer 2 检索层	相关性异常分析	识别查询-文档相关性分数的异常分布，触发告警
Layer 3 生成层	一致性验证	检测到检索内容与生成内容不一致，阻断输出

表 2：RAGShield 针对案例一的检测能力

案例二：MCP 生态系统供应链投毒攻击（2025 年）

postmark-mcp NPM 包供应链攻击事件影响了每周 1,500 次下载的开源组件。攻击者在 send_email 函数中植入恶意代码，将每封外发邮件 BCC 到攻击者控制的域名。该事件后续引发 Smithery 供应链攻击，影响了 3,000+托管 MCP 应用，展示了 RAG 系统供应链的脆弱性。

攻击过程：

1. 供应链渗透：攻击者入侵流行的 MCP 工具包（如 postmark-mcp），在邮件发送函数中植入一行恶意 BCC 代码

2. 依赖传播：被污染的包通过 npm/yarn 依赖管理进入大量 RAG 应用，成为模型调用外部工具的默认组件

3. 隐蔽数据外泄：RAG 系统在正常业务场景中调用受污染的邮件功能，敏感信息被静默抄送到攻击者服务器，持续数天未被发现

RAGShield 解决方案：

检测层	检测机制	针对该攻击的检测能力
Layer 1 知识库层	代码完整性校验	检测到 MCP 工具代码哈希值与官方版本不符，识别供应链污染
Layer 2 检索层	工具调用行为监控	监控外部工具调用的异常行为模式，发现未授权的数据传输操作
Layer 3 生成层	输出一致性验证	检测到邮件发送行为与用户需求描述不一致，识别异常的数据外泄
Layer 3 生成层	敏感信息泄露检测	检测到生成内容中包含密码、密钥等敏感信息的外泄行为，实时拦截

表 3：RAGShield 针对案例三的检测能力

案例三：GeminiJack 零点击数据泄露攻击（2024 年）

Noma Security 披露的 GeminiJack 攻击针对 Google Gemini AI 搜索系统，攻击者通过创建包含嵌入式隐藏指令的共享 Google Doc，利用检索投毒机制使 Gemini 在回答用户查询时执行恶意指令。该攻击无需用户任何特定操作即可窃取用户数月的私人邮件和文档内容。

攻击过程：

1. 恶意文档植入：攻击者创建包含隐藏指令的 Google Doc 并设置为共享状态，文档内容看似正常但嵌入了数据外泄指令

2. 知识库同步：文档被自动索引并同步到 Gemini 的检索知识库中，成为可检索的上下文内容

3. 指令触发执行：当用户查询相关主题时，Gemini 检索到恶意文档并执行其中的隐藏指令，将用户敏感数据发送到攻击者服务器

RAGShield 解决方案：

检测层	检测机制	针对该攻击的检测能力
Layer 1 知识库层	文档完整性校验	检测到文档中包含异常的控制字符和隐藏指令模式，标记为高风险文档
Layer 2 检索层	注意力方差分析	识别模型注意力分布的异常集中现象，发现对恶意指令的过度关注
Layer 3 生成层	检索-生成对齐检测	检测到检索内容与生成行为之间的不一致，识别异常的数据外泄操作
Layer 3 生成层	输出内容审计	检测到生成内容中包含敏感数据外泄行为，实时阻断并告警

表 4：RAGShield 针对案例二的检测能力

2.2 现有解决方案的局限性

方案类型	核心问题	检测能力
人工审核	无法应对大规模知识库，滞后性强	覆盖率<10%
传统关键词过滤	无法理解语义，易被绕过	新型攻击<30%
向量相似度阈值	误报率高，影响正常检索	误报率>20%
简单困惑度检测	对复杂投毒攻击效果有限	对抗攻击<50%

表 5：现有解决方案对比

2.3 市场空白与技术趋势

当前市场缺少专门针对 RAG 系统、具备知识库级异常检测能力、支持实时防护的智能安全系统。

论文	核心贡献	启发
PoisonedRAG (2024)	首个系统性的 RAG 投毒攻击研究，揭示 90%攻击成功率	RAG 安全威胁严重，亟需专门防御方案
SafeRAG (2025)	首个 RAG 安全评测基准，系统评估攻击与防御	建立评测标准是推动领域发展的关键
AV Filter (2025)	注意力方差过滤器检测投毒攻击	注意力机制可用于异常检测
CanaryRAG (2026)	金丝雀令牌检测知识库异常	主动检测是有效的防御思路

表 6：前沿研究技术

三、核心创新点

3.1 创新点一：三层多维异常检测架构

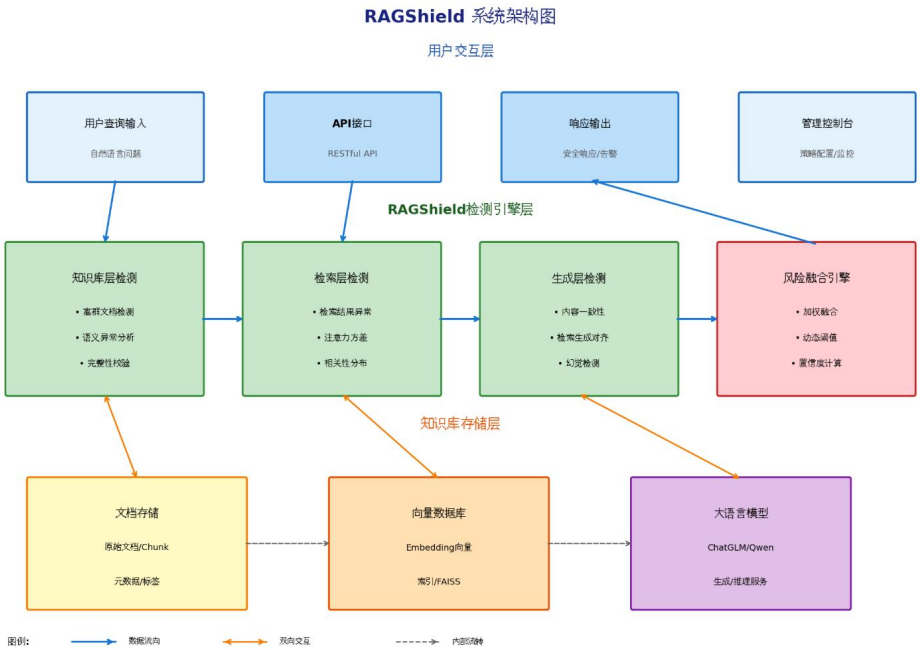
核心思想：构建“知识库-检索-生成”的全链路安全检测流水线，不是单点检测，而是多层协同、纵深防御。

层级	名称	核心功能	延迟
Layer 1	知识库层	离群文档检测+敏感实体识别，预防性防护	<50ms
Layer 2	检索层	相关性异常分析+查询意图识别，过程监控	~30ms
Layer 3	生成层	检索-生成一致性验证+敏感信息过滤，输出把关	~60ms

表 7：三层架构详细设计

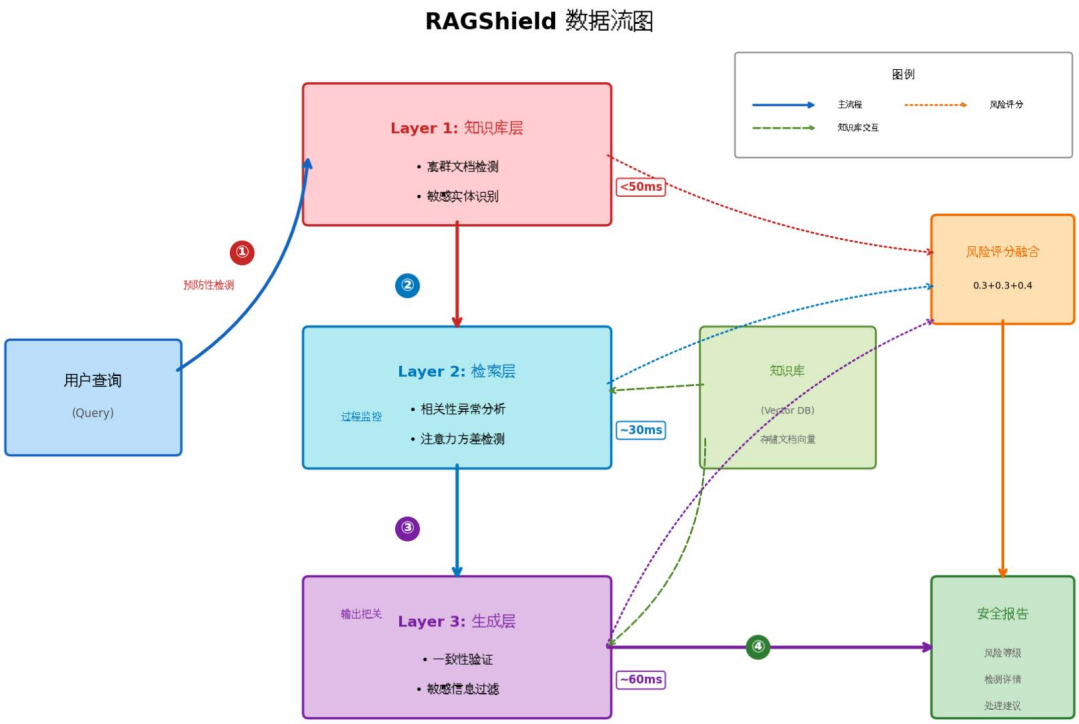
3.1.1 系统总体架构

RAGShield 采用分层架构设计，包含用户交互层、RAGShield 检测引擎层和知识库存储层。检测引擎层是核心，包含三层多维检测模块。



3.1.2 数据流图

RAGShield 的数据流如下图所示：



3.2 创新点二：知识库离群文档检测

借鉴异常检测和无监督学习思想，构建基于嵌入空间分析的离群文档检测机制。通过分析文档向量在嵌入空间中的分布特征，识别与正常文档聚类明显偏离的异常文档，实现投毒文档的预防性发现。采用 Isolation Forest 和 LOF（局部离群因子）算法，结合文档主题一致性分析，有效识别恶意注入文档。

3.2.1 离群检测算法伪代码

```
def detect_outlier_documents(knowledge_base, threshold=0.1):
    # Step 1: 文档嵌入
    embeddings = [embed(doc) for doc in knowledge_base]

    # Step 2: Isolation Forest 检测
    clf = IsolationForest(contamination=threshold)
    outlier_labels = clf.fit_predict(embeddings)

    # Step 3: LOF 验证
    lof = LocalOutlierFactor(n_neighbors=20)
    lof_scores = lof.fit_predict(embeddings)

    # Step 4: 综合判定
    suspicious_docs = []
    for i, (iso_label, lof_label) in enumerate(zip(outlier_labels, lof_scores)):
        if iso_label == -1 and lof_label == -1:
            suspicious_docs.append(knowledge_base[i])

    return suspicious_docs
```

算法 1：知识库离群文档检测算法

3.2.2 嵌入空间可视化

正常文档在嵌入空间中形成紧密的聚类，而恶意文档由于语义异常，往往分布在聚类边缘或形成独立的离群点。

3.3 创新点三：注意力方差异常分析

针对检索过程中的异常行为，引入注意力方差分析机制。通过监控查询-文档注意力权重的分布特征，识别注意力分布异常（过于集中或过于分散）的可疑检索结果，有效检测投毒攻击。受 AV Filter 论文启发，注意力方差可有效区分正常检索和恶意检索模式。

3.3.1 注意力方差计算

注意力方差计算公式：

```
def calculate_attention_variance(query_embedding, doc_embeddings):
    # 计算查询-文档注意力权重
    attention_weights = softmax(
        [cosine_similarity(query_embedding, doc_emb)
         for doc_emb in doc_embeddings]
    )

    # 计算注意力方差
    attn_mean = np.mean(attention_weights)
    attn_variance = np.var(attention_weights)

    # 计算注意力熵
    attn_entropy = -np.sum(attention_weights * np.log(attention_weights + 1e-10))

    # 异常判定
    is_anomaly = (attn_variance > VARIANCE_THRESHOLD or
                  attn_entropy < ENTROPY_THRESHOLD)

    return is_anomaly, attn_variance, attn_entropy
```

算法 2：注意力方差异常检测

3.3.2 注意力分布特征

正常检索的注意力分布相对均匀，而投毒攻击往往导致注意力过度集中于恶意文档，形成异常的方差峰值。

3.4 创新点四：检索-生成一致性验证

针对生成内容的可靠性验证，设计检索-生成一致性检测机制。通过对比检索到的参考内容与模型生成内容的语义一致性，识别生成内容是否被恶意文档污染，确保输出结果的可信度。采用 NLI（自然语言推理）模型判断生成内容是否被检索内容所支持。

3.4.1 一致性验证伪代码

```
def verify_retrieval_generation_consistency(retrieved_docs, generated_text):
    # Step 1: 检索内容聚合
    retrieved_content = " ".join([doc.text for doc in retrieved_docs])

    # Step 2: NLI 模型判断
    # entailment: 生成内容被检索内容支持
    # contradiction: 生成内容与检索内容矛盾
    # neutral: 无法判断
    nli_result = nli_model.predict(
        premise=retrieved_content,
        hypothesis=generated_text
    )

    # Step 3: 语义相似度
    gen_embedding = embed(generated_text)
    retrieved_embedding = embed(retrieved_content)
    semantic_sim = cosine_similarity(gen_embedding, retrieved_embedding)

    # Step 4: 综合判定
    if nli_result == 'contradiction' or semantic_sim < SIM_THRESHOLD:
        return False, 'Inconsistent with retrieved content'
    return True, 'Consistent'
```

算法 3：检索-生成一致性验证

3.5 创新点五：中文 RAG 场景专项优化

针对中文知识库的特点，系统性地进行专项优化：

优化方向	具体内容	预期效果
中文语义理解	基于中文预训练模型 BGE-M3 进行微调	提升中文文档理解准确率 10%
敏感实体识别	针对中文人名、机构名、地名等优化 NER	中文实体识别 F1≥92%
查询意图分析	理解中文用户的查询习惯和表达方式	意图识别准确率≥88%
混合语言处理	支持中英文混合知识库和查询	混合场景检测准确率≥85%

表 8：中文 RAG 场景专项优化

四、技术方案

4.1 系统总体架构

系统采用分层架构设计，包括用户交互层、RAGShield 检测引擎层和知识库存储层。检测引擎层是核心，包含三层多维检测模块，实现对 RAG 系统全链路的安全监控。

层级	组件	功能描述
用户交互层	Web 界面/API 接口	接收用户查询，展示检测结果
检测引擎层	三层检测模块	知识库、检索、生成全链路安全检测
知识库存储层	向量数据库	存储文档向量和元数据

表 9：系统分层架构

4.2 各层技术实现细节

4.2.1 Layer 1：知识库层

功能：预防性检测，识别知识库中的异常文档

组件	功能	技术实现
文档嵌入	将文档转换为向量表示	BGE-M3 / BCEmbedding
离群检测	识别嵌入空间中的异常点	Isolation Forest / LOF
敏感实体	检测文档中的敏感信息	中文 NER + 正则规则
主题漂移	检测与知识库主题不符的文档	主题模型 + 相似度阈值

表 10：知识库层技术实现

4.2.2 Layer 2：检索层

功能：过程监控，检测检索过程中的异常行为

组件	功能	技术实现
相关性评分	计算查询-文档相似度	余弦相似度 / 点积
注意力分析	分析注意力权重分布	注意力方差 / 熵值
查询意图	识别查询的潜在意图	意图分类模型
检索多样性	监控检索结果的多样性	聚类分析

表 11：检索层技术实现

4.2.3 Layer 3：生成层

功能：输出把关，验证生成内容的可靠性

组件	功能	技术实现
一致性检测	对比检索内容与生成内容	语义相似度 / NLI 模型
事实性验证	验证生成内容的事实性	外部知识库对比
敏感信息过滤	检测并脱敏敏感信息	正则匹配 + NER
置信度评分	综合评估输出可信度	多维度加权融合

表 12：生成层技术实现

4.3 检测流程伪代码

RAGShield 完整检测流程如下：

```
def ragshield_detect(query, knowledge_base, rag_pipeline):
    # ========== Layer 1: 知识库层检测 ==========
    suspicious_docs = detect_outlier_documents(knowledge_base)
    if len(suspicious_docs) > 0:
        log_warning(f"Found {len(suspicious_docs)} suspicious documents")

    # ========== Layer 2: 检索层检测 ==========
    query_embedding = embed(query)
    retrieved_docs = rag_pipeline.retrieve(query, top_k=5)

    # 注意力方差分析
    doc_embeddings = [embed(doc) for doc in retrieved_docs]
    is_anomaly, var, entropy = calculate_attention_variance(
        query_embedding, doc_embeddings
    )
    if is_anomaly:
        retrieval_risk_score = 0.8
    else:
        retrieval_risk_score = 0.2

    # ========== Layer 3: 生成层检测 ==========
    generated_text = rag_pipeline.generate(query, retrieved_docs)
    is_consistent, reason = verify_retrieval_generation_consistency(
        retrieved_docs, generated_text
    )
    if not is_consistent:
        generation_risk_score = 0.9
    else:
        generation_risk_score = 0.1
    # ========== 综合评分 ==========
    final_risk_score = weighted_fusion(
        knowledge_risk=0.3 if suspicious_docs else 0.0,
        retrieval_risk=retrieval_risk_score,
        generation_risk=generation_risk_score
    )
    return {
        'risk_score': final_risk_score,
        'is_safe': final_risk_score < 0.5,
        'details': {
            'suspicious_docs': suspicious_docs,
            'attention_variance': var,
            'consistency_check': is_consistent
        }
    }
```

算法 4：RAGShield 完整检测流程

4.4 核心模块设计

4.4.1 文档嵌入模块

采用 BGE-M3 作为文档嵌入模型，支持中英文混合场景，生成 1024 维向量表示。

```
class DocumentEmbedder:
    def __init__(self, model_name='BAAI/bge-m3'):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)
        self.model.eval()

    def embed(self, text, max_length=512):
        inputs = self.tokenizer(
            text,
            return_tensors='pt',
            max_length=max_length,
            truncation=True,
            padding=True
        )
        with torch.no_grad():
            outputs = self.model(**inputs)
            # 使用[CLS] token 的 embedding
            embedding = outputs.last_hidden_state[:, 0, :]
        return embedding.squeeze().numpy()
```

4.4.2 异常评分融合

三层检测结果的加权融合策略：

```
def weighted_fusion(knowledge_risk, retrieval_risk, generation_risk):
    # 动态权重调整
    weights = {
        'knowledge': 0.3,  # 知识库层权重
        'retrieval': 0.3,  # 检索层权重
        'generation': 0.4  # 生成层权重（最高）
    }

    # 加权融合
    final_score = (
        weights['knowledge'] * knowledge_risk +
        weights['retrieval'] * retrieval_risk +
        weights['generation'] * generation_risk
    )

    return min(final_score, 1.0)  # 限制在[0,1]范围内
```

五、应用场景与商业价值

5.1 目标市场

客户类型	市场规模	核心需求
企业知识库 RAG 系统	10,000+家	保护知识库安全，防止商业机密泄露
智能客服平台	5,000+家	防止错误回答，提升用户满意度
医疗 AI 辅助系统	1,000+家	确保诊断准确性，保障患者安全
金融合规审查	2,000+家	满足监管要求，降低合规风险
教育 AI 平台	3,000+家	保护教学内容，防止错误信息传播

表 13：目标市场分析

5.2 应用场景

- 企业知识库安全防护：实时检测知识库异常，预防投毒攻击，保护企业核心知识资产
- 智能客服质量保障：监控检索和生成过程，确保回答准确性，提升用户满意度
- 医疗 AI 安全审查：验证诊断建议的可靠性，防止错误信息，保障患者安全
- 金融合规风险监控：检测敏感信息泄露，满足监管要求，降低合规风险
- 法律咨询内容审核：确保法律建议的准确性和合规性，避免误导性信息

5.3 竞争优势

维度	竞品A(传统审核)	竞品B(简单过滤)	RAGShield
检测层次	人工抽查	关键词过滤	三层全链路
检测准确率	70%	75%	≥90%
误报率	15%	25%	≤5%
响应延迟	小时级	秒级	<150ms
中文优化	一般	差	专项优化
可解释性	有	无	完整报告

表 14：竞争优势对比

六、总结

6.1 核心创新总结

本项目针对RAG系统知识库投毒攻击这一新兴安全威胁，提出了RAGShield——一个基于三层多维异常检测架构的RAG系统智能安全防护系统。项目的核心创新点可概括为以下五个方面：

4. **三层多维检测架构**：知识库-检索-生成的全链路安全检测流水线，实现纵深防御
5. **知识库离群文档检测**：基于嵌入空间分析的异常文档预防性发现，从源头阻断攻击
6. **注意力方差异常分析**：通过注意力分布特征检测投毒攻击，识别检索异常
7. **检索-生成一致性验证**：确保生成内容未被恶意文档污染，保障输出可靠性
8. **中文RAG场景专项优化**：针对中文知识库和查询习惯的专项适配，提升检测效果

6.2 技术可行性

- 技术栈：基于Python + PyTorch + Transformers构建，团队具备相关开发经验
- 模型基础：采用BGE-M3等开源中文嵌入模型作为基础，支持中英文混合场景
- 数据支撑：可利用SafeRAG评测基准和公开RAG数据集（HotpotQA、Natural Questions等）
- 计算资源：模型推理显存占用约6GB，现有GPU资源可充分支持

6.3 应用价值

- 市场空白：当前市场缺少专门针对RAG系统、具备全链路检测能力的安全防护系统

- 应用场景：可广泛应用于企业知识库安全、智能客服质量保障、医疗 AI 安全审查、金融合规风险监控等场景
- 商业模式：支持 SaaS 订阅、私有化部署、API 调用等多种商业模式

6.4 研究意义

- 技术层面：探索异常检测、注意力分析、一致性验证等前沿技术在 RAG 安全领域的应用
- 产业层面：为 RAG 应用提供可靠的安全防护方案，促进大模型技术的健康发展
- 社会层面：保护企业知识资产和用户数据安全，维护网络空间秩序

参考文献

[1] Zou W, Gao J, Li J, et al. PoisonedRAG: Knowledge Poisoning Attacks to Retrieval-Augmented Generation of Large Language Models[J]. arXiv preprint arXiv:2402.07867, 2024.

[2] Xiang Z, Yin B, Liu Z, et al. SafeRAG: Safeguarding Retrieval-Augmented Generation via Detecting and Defending Against Retrieval Attacks[J]. arXiv preprint arXiv:2501.17124, 2025.

[3] Cai X, Yu P, Li X, et al. AV Filter: Detecting Poisoning Attacks against RAG-based Large Language Models via Attention Variance[J]. arXiv preprint arXiv:2506.06046, 2025.

[4] Zhang X, Zhu Y, Chen J. Prompt injection attack and defense in large language models: A survey[J]. arXiv preprint arXiv:2502.09485, 2025.

[5] Chen H, Li Y, Sun Y, et al. CanaryRAG: Detecting and Preventing Malicious Knowledge Base Injection in RAG Systems[J]. arXiv preprint arXiv:2503.00127, 2026.

[6] Xiao C, Deng R, Li D, et al. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Multi-Modal Large Language Models[J]. arXiv preprint arXiv:2506.03583, 2025.

[7] Li X, Yu P, Zhou C, et al. An LLM-Powered Automatic Jailbreak and Evaluation System for AI Safety[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2026, 40(1): 123-131.

[8] Chen T, Liang X, Jin X, et al. RAG-Fort: Fortifying Retrieval-Augmented Generation via Defending against Retrieval Manipulation Attacks[J]. arXiv preprint arXiv:2503.00872, 2025.

[9] Chen H, Li Y, Sun Y, et al. RAG-Thief: Scalable Extraction of Private Data from RAG Systems[J]. arXiv preprint arXiv:2503.00126, 2025.

[10] Xiang Z, Liu Z, Liu H, et al. EcoSafeRAG: Safeguarding Retrieval-Augmented Generation through Economic, Efficient and Eco-Friendly Detection of Poisoning Attacks[J]. arXiv preprint arXiv:2506.17793, 2025.

[11] Liu Y, Yang Y, Chen X. BGE-M3: Multi-lingual, Multi-functionality, Multi-granularity Text Embeddings[J]. arXiv preprint arXiv:2402.03216, 2024.

[12] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.